



7.5 文件传送协议

- 文件传输协议（File Transfer Protocol, FTP）通过FTP程序（服务器程序和客户端程序）在Internet上实现远程文件的传输，是因特网上使用得最广泛的文件传送协议。
- FTP实际上就是将各种类型的文件都放在FTP服务器中，用户计算机上要安装一个客户端FTP服务程序，通过这个程序实现对FTP服务器的访问。
- 当通过FTP客户端程序登录FTP服务器时，要求正确回答用户名和口令，才能取得访问权。FTP的主要功能是减少或消除在不同操作系统下处理文件的不兼容性。



- **FTP**是Internet上的一套传输文件的通信标准，**FTP**规定了其行为规范和接口交换信息的集合。
- 使用者可以通过**FTP**程序来下载或上传文件。它的任务是通过网络将文件从一台计算机上传送到另一台计算机上，并保证其传输的可靠性，这就像在操作系统下将文件在本机磁盘之间复制一样。
- 属于应用层的协议。



匿名FTP

- 在Internet上要连接FTP服务器，大多要经过一个登录（Login）的过程，要求输入用户在该主机上登记的账号和密码。
- 若提供匿名（anonymous）FTP的服务，用户不需要主机的账号和密码即可进入FTP服务器。
- 使用匿名进入服务器时，通常只能浏览及下载文件，不能上传文件或修改服务器上的文件。但也有的服务器会提供一些目录供用户上传文件。



FTP的基本工作原理

- 在两个主机之间传送文件好像是很简单的事情。其实这往往是非常困难的。原因是众多的计算机厂商研制出的文件系统多达数百种，且差别很大。经常遇到的问题：
 - (1) 计算机存储数据的格式不同；
 - (2) 文件的目录结构和文件命名的规定不同；
 - (3) 对于相同的文件存取功能，操作系统使用的命令不同；
 - (4) 访问控制方法不同。



FTP的基本工作原理

- 文件传送协议**FTP**只提供文件传送的一些基本的服务，它使用可靠的**TCP**运输服务。
- **FTP**的主要功能是减少或消除在不同操作系统下处理文件的不兼容性。
- **FTP**使用客户/服务器方式。一个**FTP**服务器进程可同时为多个客户进程提供服务。**FTP**的服务器进程由两大部分组成：一个**主进程**，负责**接受新的请求**；另外有若干个**从属进程**，负责**处理单个请求**。



FTP的基本工作原理

- 主进程的工作步骤如下：

(1) 打开熟知端口（端口号为**21**），使客户进程能够连接上。

(2) 等待客户进程发出连接请求。



FTP的基本工作原理

- 主进程的工作步骤如下：

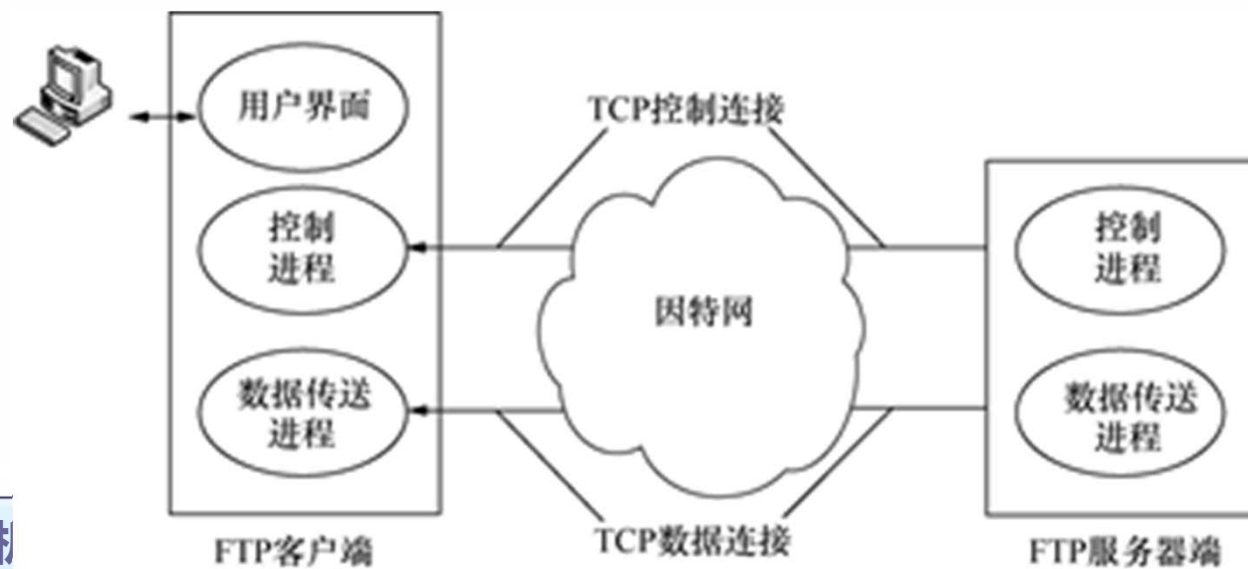
(3) 启动从属进程来处理客户进程发来的请求。从属进程对客户进程的请求处理完毕后即终止，但从属进程在运行期间根据需要还可能创建其他一些子进程。

(4) 回到等待状态，继续接受其他客户进程发来的请求。主进程与从属进程的处理是并发进行的。



FTP的基本工作原理

- **FTP**的工作情况如图所示。图中的圆圈表示在系统中运行的进程。图中的服务器端有两个从属进程：**控制进程**和**数据传送进程**。为简单起见，服务器端的主进程没有画上。在客户端除控制进程和数据传送进程外，还有一个**用户界面进程**作为和用户的接口。





FTP的基本工作原理

- 在进行文件传输时，FTP的客户和服务端之间要建立两个连接：“控制连接”和“数据连接”。
 - ⑩ 控制连接在整个会话期间一直保持打开，FTP客户所发出的传送请求通过控制连接发送给服务器端的控制进程。
 - ⑩ 控制连接并不用来传送文件。实际用于传输文件的是“数据连接”。



FTP的基本工作原理

- 服务器端的控制进程在接收到FTP客户发送来的文件传输请求后就创建“数据传送进程”和“数据连接”。
- 数据连接用来连接客户端和服务器的数据传送进程，数据传送进程实际完成文件的传送。
- 在传送完毕后关闭“数据连接”，并结束运行。



FTP的基本工作原理

- 当客户进程向服务器进程发出建立连接请求时，要寻找连接服务器进程的熟知端口，同时还要告诉服务器进程自己的端口号码，用于建立数据传送连接。
- 服务器进程用传送数据的端口与客户进程所提供的端口号码建立数据传送连接。



简单文件传送协议

- 简单文件传送协议（**Trivial File Transfer Protocol, TFTP**）是一个很小且易于实现的文件传送协议。
- **TFTP** 使用**C/S方式**和**UDP**数据报传输数据，因此 **TFTP** 需要有自己的差错改正措施。
- **TFTP** 只支持文件传输而不支持交互。**TFTP** 没有一个庞大的命令集，没有列目录的功能，也不能对用户进行身份鉴别。



简单文件传送协议

- **TFTP**的主要优点有两个:

- ⑩ 第一, **TFTP**可用于**UDP**环境;
- ⑩ 第二, **TFTP**代码所占的内存较小, 减少了开销。



简单文件传送协议

● TFTP 的主要特点如下：

- ⑩ (1) 每次传送的数据 PDU 中有 512B 的数据，但最后一次可不足 512B。
- ⑩ (2) 数据 PDU 也称为文件块 (block)，每个块按序编号，从 1 开始。
- ⑩ (3) 支持 ASCII 码或二进制传送。
- ⑩ (4) 可对文件进行读或写。
- ⑩ (5) 使用很简单的首部。



简单文件传送协议

- **TFTP** 的工作很像停止等待协议，在发送完一个文件块后就等待对方的确认，确认时应指明所确认的块编号。
 - ⑩ 发完数据后在规定时间内收不到确认就要重发数据 **PDU**。
 - ⑩ 发送确认**PDU**的一方若在规定时间内收不到下一个文件块，也要重发确认**PDU**。
 - ⑩ 这样就可保证文件的传送不致于因某一个数据报的丢失而告失败。



简单文件传送协议

- 在一开始工作时，**TFTP 客户进程**发送一个读请求**PDU**或写请求 **PDU**给 **TFTP 服务器进程**，其熟知端口号码为 **69**。
- **TFTP 服务器进程**要选择一个新的端口和 **TFTP 客户进程**进行通信。若文件长度恰好为512B的整数倍，则在文件传送完毕后，还必须在最后发送一个只含首部而无数据的数据 **PDU**。若文件长度不是512B的整数倍，则最后传送数据 **PDU** 的数据字段一定不满**512B**，这正好可作为文件结束的标志。



7.6 远程终端协议

远程终端协议TELNET

- **TELNET**是一个简单的远程终端协议，也是因特网的正式标准。
- 用户用**TELNET**就可在其所在地通过**TCP**连接注册（即登录）到远地的另一个主机上（使用主机名或**IP**地址）。
- **TELNET**能将用户的击键传到远地主机，同时也能将远地主机的输出通过**TCP**连接返回到用户屏幕。这种服务是透明的，因为用户感觉到好像键盘和显示器是直接连在远地主机上。



远程终端协议

- **TELNET**也使用**C/S**方式。在本地系统运行**TELNET**客户进程，而在远地主机运行**TELNET**服务器进程。
 - ⑩和**FTP**的情况相似，服务器中的主进程等待新的请求，并产生从属进程来处理每一个连接。
- **TELNET**使用网络虚拟终端**NVT**格式，这种格式是为了适应许多计算机和操作系统的差异。例如，对于文本中一行的结束，有的系统使用回车（**CR**），而有得系统使用换行（**LF**）。



TELNET的工作原理

远程终端协议

- **TELNET**的工作原理:

- ⑩ 客户软件把用户的击键和命令转换成**NVT**格式，并送交服务器；
- ⑩ 服务器软件把收到的数据和命令，从**NVT**格式转换成远地系统所需的格式；
- ⑩ 向用户返回数据时，服务器把远地系统的格式转换为**NVT**格式；
- ⑩ 本地客户再从**NVT**格式转换到本地系统所需的格式。

104



7.7 简单网络管理协议

网络管理的基本概念

- 网络管理包括对**硬件、软件和人**力的使用、综合与协调，以便对网络资源进行监视、测试、配置、分析、评价和控制，这样就能以合理的价格满足网络的一些需求，如实时运行性能、服务质量等。



网络管理的需求

- (1) 网络**规模快速膨胀**，网络节点的大幅度增加、跨地域，这些造成网络需要集中维护。
- (2) 网络**复杂性大幅度增加**，网络设备种类多、应用种类多、网络协议种类多。
- (3) 网络运行的**可靠性要求**，通过配置冗余的网络硬件、软件、传输介质等。
- (4) 网络**运行可用性的要求**，降低网络出故障的时间，减少网络故障的修复时间。



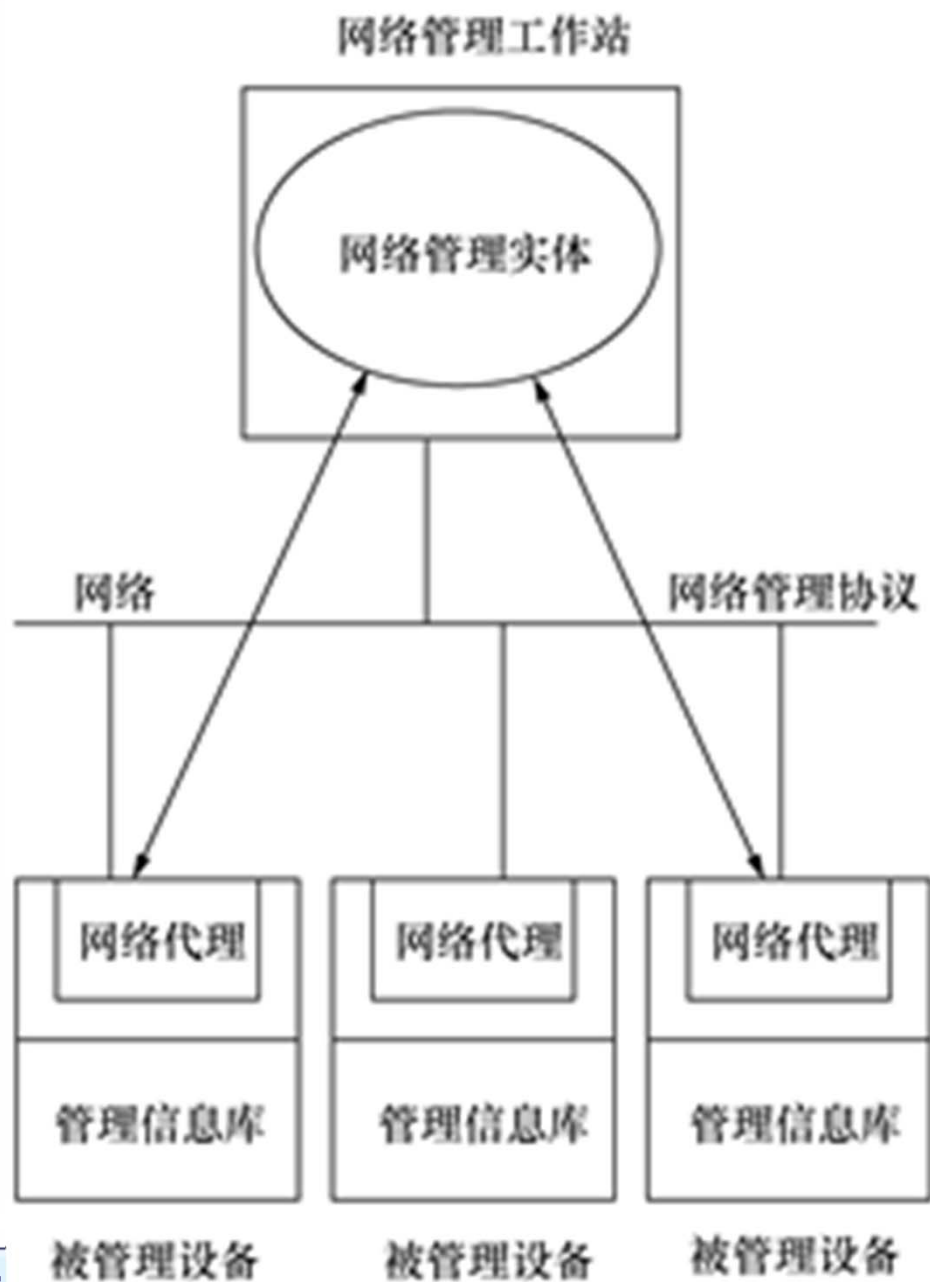
网络管理的需求

- (5) 网络**稳定性**的要求，要求提供一个运行平稳、网络性能有保障的网络。
- (6) 降低网络**运行维护成本**，集中维护模式的采用。
- (7) 网络**扩容、规划**的需要。

网络管理的**根本目标**就是为了满足运营者和用户对网络的有效性、可靠性、开放性、综合性、安全性和经济性要求。



网络管理中的主要构件





网络管理中的主要构件

● (1) 网络管理站

- 网络管理站由图形显示工作站和网络管理应用组成。
- 图形显示工作站为网络管理应用程序的运行提供存储、计算、通信和显示服务。
- 网络管理应用主要用来处理大量的数据、对事件进行响应、提供良好的人机接口界面。



网络管理中的主要构件

- (2) 被管理设备

- 被管理的设备可以是网络上的任何节点，例如，计算机、打印机、路由器、网络应用等；
- 被管理的设备通常有一个管理代理在后台运行。

- (3) 管理代理

- 管理代理向网络管理应用提供被管理设备的信息，同时也可以接收来自网络管理应用的控制信息。



网络管理中的主要构件

- (4) 网络管理协议

→ 网络管理协议用于网络管理应用和网络管理之间交换信息，定义网络管理信息的结构、数据格式等。

- (5) 网络管理信息

→ 网络管理信息是网络管理应用和网络管理代理之间交换的信息，用于监视和控制被管理的设备。



网络管理的功能

网络管理的功能

- (1) 性能管理：用于衡量和呈现网络特性的各个方面
 - 收集网络管理者感兴趣的那些变量的性能数据。
 - 分析这些数据，以判断是否处于正常（基线）水平并产生相应的报告。
 - 为每个重要的变量确定一个合适的性能阈值，超过该阈值就意味着出现了应该注意的网络故障。



网络管理的功能

- (2) 故障管理：用于检测、记录网络故障并通知给用户，尽可能自动修复网络故障以使网络能有效地运行。
 - 检测被管理对象的差错或接收差错事件报告。
 - 在紧急情况下启用备用资源用于服务。
 - 创建和维护差错日志库并对其进行分析。
 - 对故障进行诊断测试、跟踪、识别和定位。
 - 对有故障资源进行更换、修复，或采取其他恢复措施使其重新投入使用。



网络管理的功能

- (3) 配置管理：用于监视网络和系统的配置信息，以便跟踪和管理不同的软硬件元素对网络操作的作用
 - 网络资源的配置及其活动状态的监视
 - 网络资源之间的关系的监视与控制
 - 加入新资源，删除旧资源，定义新的管理对象
 - 识别管理对象，给每个对象分配名字
 - 初始化对象，启动、关闭对象
 - 管理各个对象之间的关系
 - 改变管理对象的参数



网络管理的功能

- (4) 计费管理：用于对用户使用情况网络资源的情况进行记录，并根据预定的计算方法和收费标准来核算用户的通信费用。

→统计网络效益数据，以便确定不同时期和不同时段的费用率。

→根据用户使用的特定业务和计费方式，确定和计算各个用户的费用。

→账务管理、分类和稽查功能。



网络管理的功能

- (5) **安全管理**: 用于控制对网络资源的访问, 以保证网络不被有意或无意地侵害, 并保证敏感信息不被那些未授权的用户访问。
 - ➔ 标识重要的网络资源 (包括系统、文件和其他实体)。
 - ➔ 确定重要的网络资源和用户集间的映射关系。
 - ➔ 监视对重要网络资源的访问。
 - ➔ 记录对重要网络资源的非法访问。



简单网络管理协议SNMP

SNMP

- 若要管理某个对象，就必然会给该对象添加一些软件或硬件，但这种“添加”必须对原有对象的影响尽量小些。
- 简单网络管理协议**SNMP**中的管理程序和代理程序按**C/S**方式工作。
- **SNMP**发布于**1988**年，**IETF**在**1990**年将**SNMP**定为因特网的正式标准，以后又有了新版本**SNMP v2**和**SNMP v3**，因此原来的**SNMP**又称为**SNMP v1**。



SNMP

- **SNMP最重要的指导思想就是要尽可能简单。**
- **SNMP的基本功能包括监视网络性能、检测分析网络差错和配置网络设备等。**
- 在网络正常工作时，**SNMP**可实现统计、配置和测试等功能；当网络出故障时，可实现各种差错检测和恢复功能。
- 虽然**SNMP**是在**TCP/IP**基础上的网络管理协议，但也可扩展到其他类型的网络设备上。



SNMP的工作过程

SNMP的工作过程

- (1) 管理软件和代理软件开始正常运行，网络管理员把要管理的子网或者网元加入到网络管理系统中，网络管理软件根据被管理的设备和链路，描绘出整个网络结构。
- (2) 管理软件定期轮询各网元设备。
- (3) 当无法轮询某个网元设备，就修改网络拓扑图，并发出报警。



SNMP的工作过程

- (4) 管理软件也接收来自各网元发出的中断事件。
- (5) 管理软件可以根据网管人员的要求，向各网元设备发出查询、参数设备等命令。



SNMP的工作过程

- 在SNMP协议中，**get**指令用来获取对象信息，**set**指令用来设置对象属性，**trap**指令用来实现中断消息的传递。
- 在SNMP协议工作时
 - ➔ 代理设备监听**UDP端口161**进行通信：如果采用轮询方式；
 - ➔ 管理工作端监听**UDP端口162**进行通信：如果采用中断方式。



SNMP的协议数据单元和报文

- **SNMP v1**一共定义了**五种协议数据单元类型**。

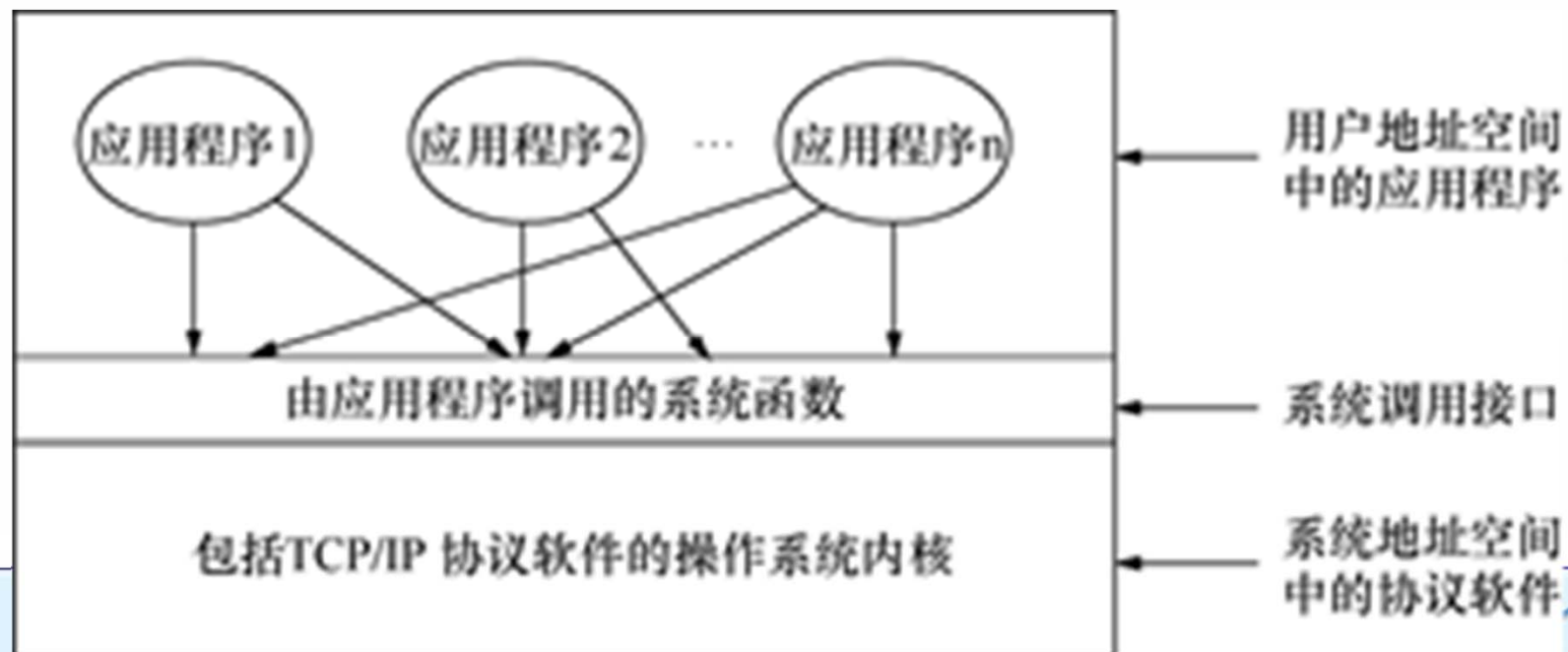
PDU 编号	PDU 名称	用 途
0	get-request	用来查询一个或多个变量的值
1	get-next-request	允许在一个 MIB 树上检索下一个变量
2	get-responses	对 get /set 报文作出响应
3	set-request	对一个或多个变量值进行设置
4	trap	向管理进程报告代理中发生的事件



7.8 应用进程跨越网络的通信

系统调用和应用编程接口

- 大多数操作系统使用系统调用（system call）的机制在应用程序和操作系统之间传递控制权。对程序员来说，每一个系统调用和一般程序设计中的函数调用非常相似，只是系统调用是将控制权传递给了操作系统。





系统调用和应用编程接口

系统调用和应用编程接口

- 当某个应用进程启动系统调用时，控制权就从应用进程传递给了系统调用接口。
- 此接口再将控制权传递给计算机的操作系统，而操作系统将此调用转给某个内部过程，并执行所请求的操作。
- 内部过程一旦执行完毕，控制权就又通过系统调用接口返回给应用进程。系统调用接口实际上就是应用进程的控制权和操作系统的控制权进行转换的一个接口，即应用编程接口（Application Programming Interface, API）。



系统调用和应用编程接口

- 目前只有几种可供应用程序使用的**TCP/IP**应用编程接口
 - **Berkeley UNIX** 操作系统中定义的一种**API**，它又称为插口接口（**socket interface**）；
 - 微软公司在其操作系统中采用了插口接口**API**，形成了一个稍有不同的**API**，并称之为**Windows Socket**。



系统调用和应用编程接口

- **Socket和API之间的区别:**

- ➔ **Socket**是应用进程和网络之间的接口，因为**Socket**既包含运输层与应用层之间的端口号，又包含机器的**IP**地址。

- ➔ **API**是应用程序和操作系统之间的接口。**API**从程序设计的角度定义了许多标准的系统调用函数。应用进程只要使用标准的系统调用函数就可得到操作系统的服务。



系统调用和应用编程接口

- 在**Socket**以上的进程是受应用程序控制的，而在**Socket**以下的**TCP**协议软件以及**TCP**使用的缓存和一些必要的变量等，则是受计算机操作系统的控制。
- 应用程序使用**TCP/IP**协议进行通信时，必须通过**Socket**与操作系统交互并请求其服务。应用程序的开发者和对**Socket**以上的应用进程具有完全的控制，但对**Socket**以下的运输层却只有少量的控制。



几种常用的系统调用

- 主要的系统调用有以下几个

- ➔ (1) **socket()**调用：用来创建一个**socket**，这个调用返回一个整数。
- ➔ (2) **bind()**调用：指定**socket**所使用的IP地址和端口号，又称“本地插口地址”。
- ➔ (3) **recvfrom()**调用：将**socket**的入队列中的下一个数据报提取出来。



几种常用的系统调用

- 主要的系统调用有以下几个

- ➔ (4) **sendto()**调用：将一个数据报从出队列中取出，并用**UDP**发送给远地机器的一个进程。远地机器的**socket**地址是从上面的**recvfrom()**调用得到的。
- ➔ (5) **close()**调用：用来关闭一个**socket**。



几种常用的系统调用

- 主要的系统调用有以下几个

→ (6) **listen()** 调用： 仅为**TCP**服务器使用的系统调用，其作用是使已经创建的**Socket**变成被动**Socket**，即监听**Socket**。监听**Socket**的用处不是和远地**Socket**建立连接，而只是等待远地客户发出的连接请求。

Listen()调用通知操作系统：服务器已做好接受连接的准备。



几种常用的系统调用

- 主要的系统调用有以下几个

- ➔ (7) **connect()**调用：通常是客户进程使用的系统调用，其功能是向远地进程（通常是服务器）请求建立连接。

- ➔ (8) **accept()**调用：是TCP服务器使用的系统调用，其作用是从入队列中提取最前面的连接要求。**accept()**调用会创建一个新的socket，叫做接受socket；此后，客户将只和这个新创建的接受socket建立连接并通信，而不再和监听socket发生联系。



几种常用的系统调用

- 主要的系统调用有以下几个

- ➔ (9) **fork()**调用：创建一个和自己完全一样的从属进程（或子进程）。
- ➔ (10) **read()**调用：读取从远地机器通过**TCP**连接传送到缓存中的数据。
- ➔ (11) **write()**调用：通过**TCP**连接将数据发送到远地机器的缓存中。



本章小结

● 内容

以Internet应用为例，介绍一些典型的应用层服务和协议，包括域名系统（DNS）、WWW（HTTP）、电子邮件（SMTP）等。

● 重点

域名系统（DNS）、WWW服务、HTTP协议、FTP、TELNET、SNMP、Socket